WHY · WHAT · HOW · CONSIDERATIONS

The Agent Builder's Playbook

AI should solve a real problem, not be AI for AI's sake. This is a business-first guide to building agents — when to build one, what it's worth, how to build it right, and what can go wrong.

Using a **Workday Time-Off Agent** as the running example.

💰

Business Case

Why this problem, ROI math, cost per request, monthly projections

⚙

Build Guide

Instruction files, tool design, architecture, testing

⚠

Risks & Trust

Cost, accuracy, safety, security, bias, latency, governance

PART I

Why Build an Agent?

Start with the business problem. If there's no real pain, there's no reason to build.

WHY

The Problem Worth Solving

Before you write a line of code, ask: what pain does this eliminate? Who feels it? How often?

RUNNING EXAMPLE

Time-Off Requests Are Broken

- ✓ **Employees** dig through confusing Workday forms, check the wrong leave type, forget about holidays, submit incorrectly, wait days for answers
- ✓ **HR teams** answer the same PTO questions hundreds of times, manually check balances, chase down approvals, lose hours every week
- ✓ **Managers** get approval emails with no context — no team calendar, no coverage info, no conflict check
- ✓ **The company** pays $4.67 per request in HR time, deals with policy errors, and employees avoid the process altogether

The Goal

An AI assistant that lets employees say *"I want to take July 14-18 off"* and handles everything: checks their balance, identifies holidays, confirms with them, and routes the approval — in 30 seconds instead of 3 days.

🔎 **The acid test:** If nobody would use this voluntarily, don't build it. The time-off agent passes because employees *hate* the existing process and would genuinely prefer a chat interface that just works.

WHY

Do I Actually Need an Agent?

Not everything needs AI. An agent only makes sense when the task is conversational, multi-step, and varies by user. Otherwise you're paying for AI you don't need.

The Quick Check — 5 Questions

Does what the user wants change a lot between sessions?	YES = AGENT Agent figures out intent
Are there 3+ actions to choose between based on context?	YES = AGENT Agent picks the right tools
Does the task need multiple steps with reasoning in between?	YES = AGENT Agent thinks through steps
Is a back-and-forth conversation better than a form?	YES = AGENT Agent talks naturally
Can the task go wrong in ways that need flexible recovery?	YES = AGENT Agent adapts gracefully

When NOT to Use an Agent (Save Your Money)

High-volume, same-every-time

Expense categorization → Use a simple classifier

Fixed step-by-step process

New hire provisioning → Use a workflow engine

Needs millisecond response

Real-time fraud detection → Use rules + ML models

Simple data entry

"Update my address" → Use a form

The Cost-Benefit Gut Check

What You Pay		What You Get
AI processing fees + Engineering time + Response latency + Ongoing testing	VS	HR ticket deflection + Faster resolutions + Fewer errors + Employee satisfaction

The agent wins when the current process is confusing, involves hidden rules, mistakes are costly, and people do it infrequently enough that they never learn the system.

PART II

What's It Worth?

Model costs are real. Before building, know the math: what each request costs, what you save, and when it pays for itself.

VALUE

Cost & Token Estimation

"How much will this cost per employee?" is the #1 question from customers. Here's how to answer it.

What Are Tokens?

Tokens are the units of text the AI reads and writes. Think of them as word-fragments: **1 token ≈ ¾ of a word**. A 100-word message is ~130 tokens. You pay for what goes in (your prompt + conversation) and what comes out (the AI's reply). That's the whole pricing model.

Claude Model Pricing (June 2025)

MODEL	BEST FOR	INPUT (PER 1M TOKENS)	OUTPUT (PER 1M TOKENS)
Claude Haiku	AGENTS & HIGH-VOLUME	$1.00	$5.00
Claude Sonnet	BALANCED QUALITY/COST	$3.00	$15.00
Claude Opus	COMPLEX REASONING	$5.00	$25.00

Pro tip: *Prompt caching (reusing the same system prompt across calls) cuts input costs by ~90%. Most agent setups get this automatically.*

RUNNING EXAMPLE

Time-Off Agent: Token Breakdown per Request

A typical "I want to take July 14-18 off" conversation: **3-4 back-and-forth turns**, **3-5 tool calls**.

STEP	TOKENS IN	TOKENS OUT	WHAT'S HAPPENING
Turn 1	~1,800	~200	Instructions + user message → agent decides to look up profile
Tool calls	~2,200	~350	`get_employee_profile` + `get_leave_balances` results come back
Turn 2	~2,800	~300	Agent checks for date conflicts and holidays
Tool calls	~3,200	~250	`check_date_conflicts` + `get_company_holidays` results
Turn 3	~3,800	~400	Agent shows summary, asks "ready to submit?"
Turn 4	~4,200	~200	User says "yes" → `submit_time_off_request` → done
TOTAL	**~18,000**	**~1,700**	**One complete time-off request**

Tokens in grow each turn because the full conversation is resent. Caching the ~800-token system prompt saves money on every call after the first.

Cost per Request

$0.03
Haiku

$0.08
Sonnet

$0.13
Opus

~$0.01
Haiku + caching

Agent vs. Manual Process — The Real Comparison

Manual (HR Specialist)

- HR specialist cost: ~$35/hr fully loaded
- Avg time per request: 8 min
- Cost per request: **$4.67**
- Response time: **4-24 hours**
- 10K employees × 2/mo = **$93,400/mo**

AI Agent (Haiku + caching)

- Cost per request: **$0.01**
- Response time: **3-5 seconds**
- 10K employees × 2/mo = **$200/mo**
- Available 24/7, zero backlog
- **99.8% cost reduction**

Even at 30% deflection (not all requests go through the agent), the savings are dramatic. The agent pays for itself in the first week.

Monthly Cost at Scale

Assuming **~2 time-off requests per employee per month**:

COMPANY SIZE	REQUESTS/MONTH	HAIKU + CACHE	SONNET	MANUAL (HR)
100 employees	200	**$2**	$16	$934
1,000 employees	2,000	**$20**	$160	$9,340
10,000 employees	20,000	**$200**	$1,600	$93,400
50,000 employees	100,000	**$1,000**	$8,000	$467,000

💡 **Cost optimization:** (1) Use Haiku for most agent tasks — fast, cheap, accurate enough. (2) Enable prompt caching — 90% savings on repeated instructions. (3) Summarize conversation

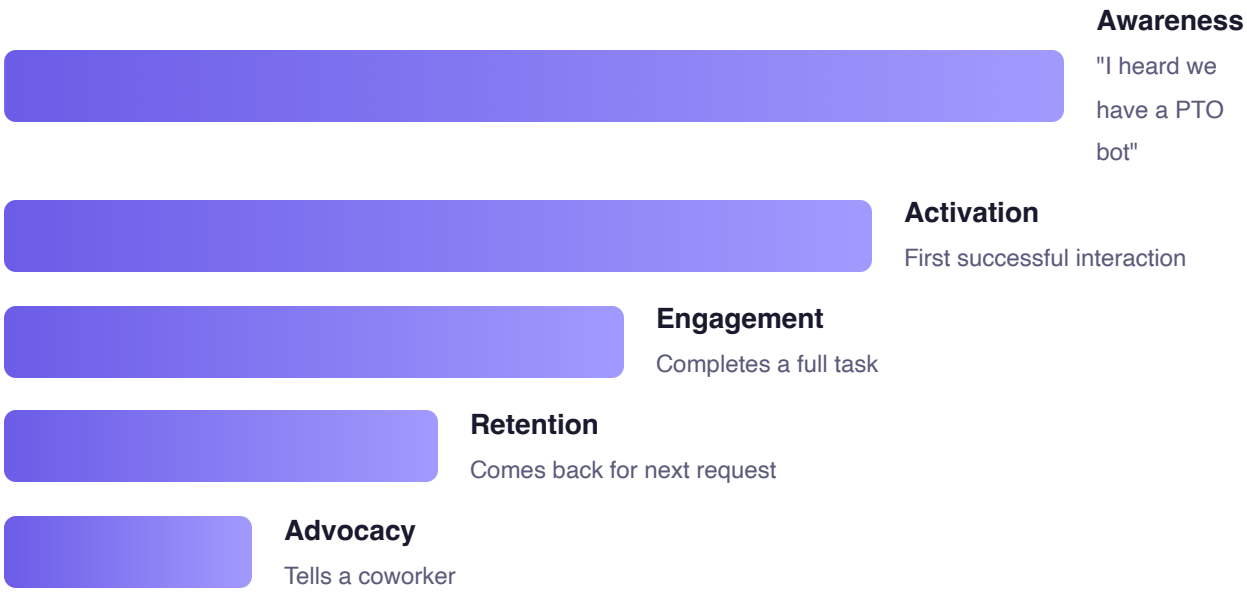
history after 8 turns. (4) Use Sonnet/Opus only for genuinely complex reasoning like policy interpretation.

VALUE

Business Metrics & Adoption

The metric that actually matters: do people come back unprompted?

The Adoption Funnel



Targets by Phase

METRIC	PILOT	GENERAL AVAILABILITY
Activation rate	≥ 50%	≥ 40%
Task completion	≥ 70%	≥ 80%
Avg conversation turns	≤ 5	≤ 4
Return rate (30d)	≥ 30%	≥ 50%
Channel shift (chat vs. old process)	≥ 10%	≥ 30%
HR ticket deflection	—	≥ 25%
NPS	≥ 50	≥ 70

VALUE

Launch Plan

Don't launch to everyone on day one. Expand in concentric circles.

Launch Timeline

WEEK 1-2

Closed Pilot

15-20 people from 2-3 teams. Watch every conversation. Daily Slack check-in. Goal: find the top 5 failure modes.

WEEK 3-4

Open Pilot

One full department. Track activation (≥50%) and completion (≥70%). Fix top 3 issues from closed pilot.

MONTH 2

Expand & Iterate

3-5 departments. Add Slack/Teams integration. Measure channel shift and HR ticket deflection.

MONTH 3+

General Availability

Org-wide rollout. HR endorsement. Deprecate redundant FAQ pages. Target: 30%+ channel shift.

PART III

How to Build It

The technical guide. From the agent's instruction file to tool design, architecture, and testing.

BUILD

The Instruction File

The `.md` file is the agent's instruction manual — the single most important file you'll write. Everything the agent does flows from this document.

What Goes In It

Identity	Who the agent is, who it serves, what it does
Context Variables	Dynamic runtime data: `{{TODAY}}`, `{{EMPLOYEE_NAME}}`, `{{EMPLOYEE_TYPE}}`
Capabilities	Every tool listed with *when* to use it, not just *what* it does
Boundaries	What the agent refuses and how it handles those requests
Decision Rules	If/then logic: contractor → unpaid only, low balance → suggest alternatives
Tone & Format	How to sound (specific, not "be professional") and exact output templates

system-prompt.md

```
# Time-Off Assistant — System Prompt

## Identity
You are a time-off assistant for {{COMPANY}}.
You help employees check leave balances, plan time off,
and submit requests through Workday.

## Tool Ordering Rules
- NEVER call submit_time_off_request without first
  calling check_date_conflicts
- ALWAYS call get_leave_balances before presenting
  a submission summary

## Boundaries — Do Not Handle
- Medical leave / FMLA → "Please contact HR directly"
- Payroll questions → "Reach out to payroll"
- Other employees' balances → "I can only access your info"

## Decision Rules
- IF contractor → only offer unpaid time off
- IF balance < request → explain, suggest alternatives
- IF dates include holidays → exclude from PTO count
```

💡 **This file is your biggest lever.** Prompt engineering isn't magic — it's writing clear instructions. A well-written .md file eliminates entire categories of bugs without writing any code.

BUILD

Tool Design

Tools are how the agent interacts with your systems. Design them like good APIs — each one does one thing well.

1

One Tool, One Job

✗ `submit_time_off(auto_check=true)`

✓ `check_conflicts()` then `submit()`

The agent should decide *whether* to check, not have it hidden.

2

Return Data, Not Decisions

✗ `→ "Employee is eligible"`

✓ `→ {eligible: true, types: [...]}`

The agent interprets. The tool just reports facts.

3

Actions Return Updated State

✗ `→ {success: true}`

✓ `→ {id: "REQ-001", balance_after: ...}`

No second call needed to see what happened.

4

Name for What, Not How

✗ `workday_api_get_v2_absence()`

✓ `get_leave_balances()`

If you swap to BambooHR tomorrow, the prompt shouldn't change.

Tool Inventory — Read vs. Write

TOOL	TYPE	RISK	NEEDS CONFIRMATION?
`get_employee_profile`	READ	None	No
`get_leave_balances`	READ	None	No
`check_date_conflicts`	READ	None	No
`get_company_holidays`	READ	None	No
`get_team_calendar`	READ	Low	No
`submit_time_off_request`	WRITE	**High**	**YES**

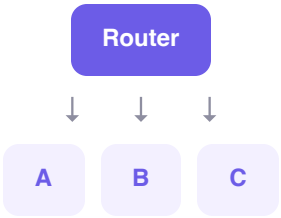
*The rule: **reads are free, writes need human confirmation.***

BUILD

Architecture: One Agent or Many?

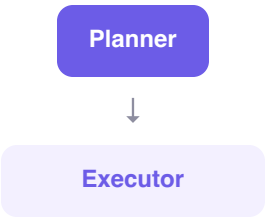
Start with one agent. Only split into multiple when you have a concrete reason — not because it sounds more sophisticated.

Common Patterns



Router + Specialists

Multiple distinct domains. "Reset password" → IT, "Check PTO" → HR.



Planner + Executor

One agent plans, another executes. Safety through separation.

Chain (Pipeline)

Sequential steps. Classify → extract → execute → format.

Supervisor + Workers

Parallel research or analysis. Fan out, collect, combine.

When to Split

FACTOR	ONE AGENT	MULTIPLE AGENTS
Number of tools	≤ 10	> 10 or different access levels
Response time needed	< 2 seconds	> 3 seconds OK
Security scopes	Same access	Different access levels
Conversation shape	Linear	Branching / parallel
Team ownership	One team	Multiple teams

🔎 **The time-off agent is a single-agent use case.** 6 tools, one user's data, linear conversation. Multi-agent would add complexity for zero benefit here.

BUILD

Agent Work Cycles

What makes an agent different from a one-shot AI call? Loops. The agent thinks, acts, sees the result, and repeats until the job is done.

🔄

Think-then-Act Core

Think → call a tool → read the result → think again → respond. **The default behavior.**

Cap: **8-10 calls/turn** Built into SDK

💬

Conversation Multi-turn

User → agent → user → agent. Memory maintained across turns.

Timeout: **30 min** Summarize after turn 8

🔁

Retry Self-correction

Try → check if it worked → retry if not. For error recovery.

Cap: **3 retries max** Then hand off to a human

RUNNING EXAMPLE

One Turn in Action

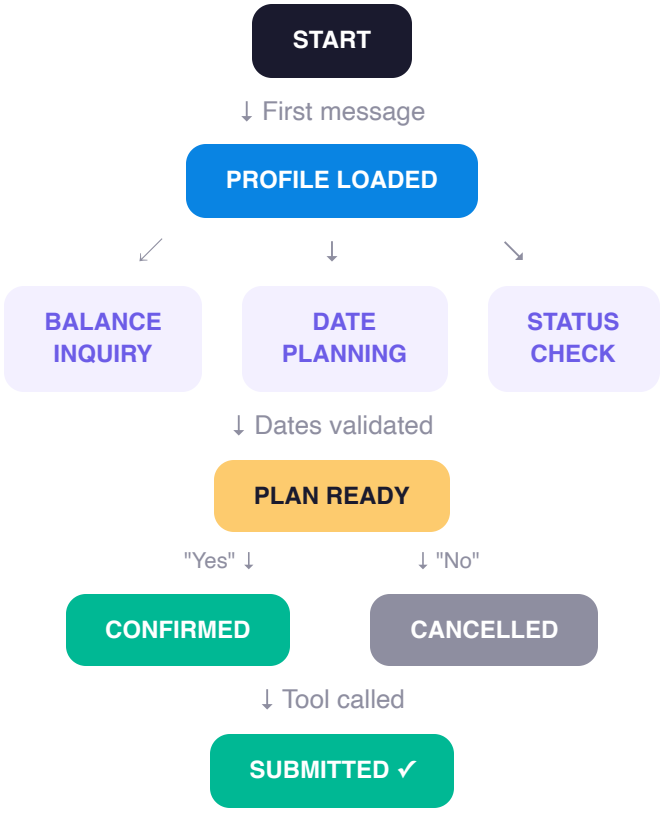
USER	"I want to take July 1-3 off"
THINK	Need to check conflicts first
TOOL	`check_date_conflicts()` → July 3 = Independence Day
THINK	Should check balance too
TOOL	`get_leave_balances()` → 12 days available
RESPOND	Summary: 2 PTO days (holiday excluded), 10 days remaining. Submit?

BUILD

Conversation Flow (State Machine)

Hard rules around flexible AI. The state machine defines what the agent *is allowed to do* at every point in the conversation — no exceptions.

Flow Chart



What This Enforces

1. `submit_time_off_request` is ONLY callable in **CONFIRMED** state
2. **PLAN_READY** requires that `check_date_conflicts` was called first
3. **CONFIRMED** requires explicit user confirmation (the agent can't assume "yes")
4. Read tools work in ALL states; write tools only work in **CONFIRMED**

BUILD

When to Ask a Human

Not every action needs human approval. But every irreversible action does.

The Approval Spectrum

Fully Autonomous	Check balance, show holidays — no risk, just go
Propose + Approve	Submit PTO request — show summary, wait for "yes"
Assist	Interpret policy ambiguity — offer guidance, let human decide
Escalate	FMLA / medical leave — hand off to HR completely

Good vs. Bad Confirmation Design

✗ Dangerous

Done! I submitted your PTO request.

✓ Safe

📋 **Time Off Request Summary**

• Type: PTO

• Dates: Jul 14 → Jul 18 (5 days, 40 hours)

• Balance after: 7 days remaining

• Approver: Jordan Park

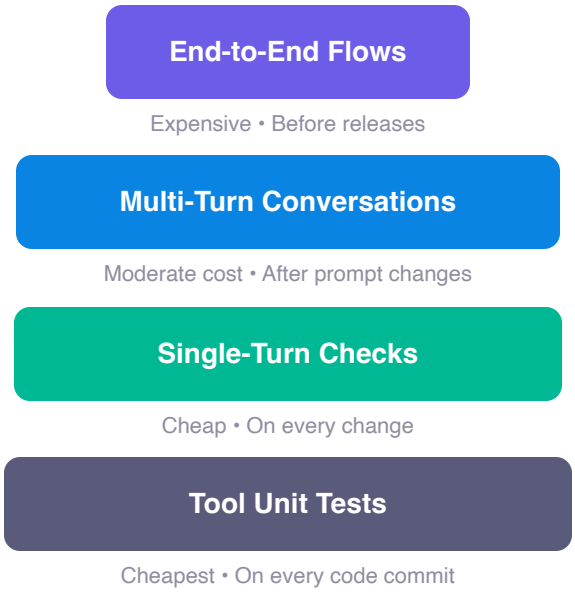
Ready to submit? *(This will notify Jordan)*

BUILD

Testing Framework

If you don't have an automated test for it, it will break and nobody will notice until a real user complains.

The Testing Pyramid



Key Quality Metrics

≥ 98% Test pass rate	**100%** Safety tests
≤ 4 Avg turns to finish	**0%** Made-up-info rate

PART IV

What Can Go Wrong

Model costs, accuracy, safety, security, bias, latency, and trust are all real concerns. Address them upfront, not after launch.

RISK

Risks & Governance

Every AI project has these seven risks. The question isn't whether they apply — it's how you mitigate each one.

💰 Cost Overruns

Token costs can surprise you. A chatty agent that doesn't summarize history will send growing context every turn, running up the bill.

✓ Use Haiku for routine tasks ($0.01/request vs $0.13)

✓ Enable prompt caching (90% input savings)

✓ Cap tool calls per turn (8-10 max)

✓ Summarize after 8 turns to control context growth

✓ Set per-user daily cost limits

🎯 Accuracy & Hallucinations

The agent might make up leave balances, invent policies, or miscount work days. In HR, wrong information has real consequences.

✓ Never let the agent state facts without calling a tool first

✓ Return raw data from tools, let the agent format it

✓ Test edge cases: holidays, weekends, zero balances

✓ Log every claim for audit — trace back to source data

✓ Run eval suite after every prompt change (target: 0% hallucination)

⚡ Latency & Performance

Each AI call takes 1-3 seconds. A 4-turn conversation with tool calls = 8-15 seconds. Users expect instant, not "processing..."

- ✓ **Stream responses (show text as it generates)**
- ✓ **Use Haiku for speed (2-3x faster than Opus)**
- ✓ **Batch tool calls where possible**
- ✓ **Show typing indicators so users know it's working**
- ✓ **Target: under 5 seconds for full response**

🛡️ Safety & Guardrails

An agent that submits PTO without confirmation, accesses other employees' data, or handles medical leave it shouldn't — that's not a bug, it's a liability.

- ✓ **Server-side rules the agent literally cannot bypass**
- ✓ **State machine: write actions only after explicit confirmation**
- ✓ **Hard boundaries in the prompt for out-of-scope requests**
- ✓ **Rate limits: cap at 5 submissions/day per employee**
- ✓ **100% pass rate on safety assertions — zero tolerance**

🔒 Security & Data Privacy

The agent sees sensitive employee data: names, balances, manager relationships. Every interaction is a potential data exposure.

✓ SSO authentication — verify identity before any data access

✓ Scope tools to the authenticated user only

✓ Strip personal info from logs (output filter layer)

✓ Never put employee data in URLs or query strings

✓ Audit trail: who accessed what, when, through which tool

⚖ Bias & Fairness

Could the agent treat contractors differently than it should? Respond differently to different names? Apply policy inconsistently?

✓ Test with diverse employee types: FT, PT, contractor

✓ Decision rules in code, not in the AI's discretion

✓ Regular audits: sample conversations across demographics

✓ Policy interpretation goes to HR, not the agent

✓ Consistent formatting for all users (same summary template)

🤝 Trust & Transparency

Employees won't use an AI they don't trust. Trust comes from accuracy, transparency, and the ability to verify. If the agent makes one wrong claim about their balance, they'll never use it again.

- ✓ **Always show source data: "Based on your Workday profile, you have 12 days remaining"**
- ✓ **Show full summary before any action — no surprises**
- ✓ **Provide a "see details" option that shows exactly what tools were called**
- ✓ **Make it easy to escalate to a human at any point**
- ✓ **Start with low-risk tasks (balance checks) before enabling submissions**
- ✓ **Publish accuracy metrics internally: "99.2% accuracy on balance lookups this month"**

Risk Profile: Time-Off Agent

RISK AREA	SEVERITY	PRIMARY MITIGATION
Cost	LOW	Haiku + caching = $0.01/request
Accuracy	MEDIUM	All facts come from tool calls, never generated
Latency	LOW	Haiku + streaming = 3-5 second responses
Safety	HIGH	State machine + server-side confirmation gate
Security	MEDIUM	SSO + scoped to authenticated user only
Bias	LOW	Policy rules in code, not AI discretion
Trust	MEDIUM	Show source data, full summary before actions

RISK

Guardrails — Layered Defense

Don't rely on any single layer. Stack them: server-side rules the agent can't bypass, prompt rules it almost always follows, output filters that catch what slips through, and monitoring that watches everything.

Layer 1 — Server-Side Rules (Cannot Be Bypassed)

Hard-coded in your backend. The agent literally cannot violate these.

- Balance check before submit
- Rate limiting (5/day)
- Confirmation flag required

Layer 2 — Prompt Rules (~97-99% Reliable)

Instructions in the system prompt. Followed almost always, but not guaranteed.

- Show summary before submit
- Don't assume confirmation
- Mention holidays proactively

Layer 3 — Output Filters (Safety Net)

Check the agent's response before the user sees it.

- Hide personal info (SSN, etc.)
- Strip raw code from responses
- Sanitize any links

Layer 4 — Monitoring (After the Fact)

Async review. Flag sessions for human inspection.

- High tool call count (>15)
- Rapid submissions
- Error rate spikes

RISK

Monitoring

If you can't see what the agent did and why, you can't fix it when it breaks. Log everything.

Structured log per turn

```
{
  "session_id": "sess_abc123",
  "turn": 3,
  "state": "DATE_PLANNING",
  "tools_called": [
    "check_date_conflicts",
    "get_leave_balances"
  ],
  "latency_ms": 1150,
  "tokens": { "input": 2400, "output": 380 },
  "cost_usd": 0.004,
  "guardrails": {
    "personal_info_redacted": false,
    "scope_violation": false
  }
}
```

Three Dashboards

⚡

Real-time (Ops)

- Active sessions
- Response latency p95
- Tool error rate
- Daily token spend

📈

Daily (Product)

- Sessions started / completed
- Task completion rate
- Cost per request trend
- Escalation rate

🔍

Weekly (Quality)

- Test suite pass rate
- Sampled conversation scores
- Made-up-info incidents
- User-reported issues

⚠

Common Mistakes

Patterns that kill agent projects — from both the business and technical side.

1

AI for AI's Sake

Built an agent because "we should have AI." No clear problem or ROI.

Fix: Start from the pain. If nobody's frustrated, don't build it.

2

"Just Add Another Agent"

7 agents for a task that needs 1. Complexity explodes.

Fix: Start with 1. Split only for concrete reasons.

3

Skipping the Cost Math

Launched with Opus, got a $50K/mo surprise bill.

Fix: Run token estimates before you build. Start with Haiku.

4

"The AI Will Figure It Out"

No state machine, no guardrails, the prompt does everything.

Fix: Use hard-coded rules for decisions that should never vary.

5

Over-Automation

Agent submits, modifies, cancels — all without asking.

Fix: Every irreversible action gets its own confirmation.

6

The Testing Desert

"It works when I test it manually."

Fix: Write the test BEFORE the feature. Run evals on every prompt change.

7

"Works on Demo Data"

Beautiful demo. Breaks on first real user.

Fix: Test with messy data early. Unicode names, negative balances, edge cases.

8

Ignoring Trust

Agent is accurate but employees don't use it. Never showed its work.

Fix: Show source data, cite tools, make escalation easy. Trust is earned.

✓

Ship Checklist

Use this for every new agent project.

- [ ] **Why:** Clear business problem identified, users actually want this
- [ ] **Why:** Agent is the right solution (not a form, classifier, or workflow)
- [ ] **Value:** Cost-per-request estimated, ROI math checks out
- [ ] **Value:** Adoption metrics defined, launch plan in phases
- [ ] **Build:** Instruction file covers identity, capabilities, boundaries, tone
- [ ] **Build:** Tools do one thing each, reads vs. writes separated
- [ ] **Build:** Architecture justified (one agent vs. many)
- [ ] **Build:** State machine with explicit transitions and tool permissions
- [ ] **Build:** Human approval levels defined for every action
- [ ] **Build:** Testing framework with all 4 pyramid levels
- [ ] **Risk:** Cost, accuracy, latency, safety, security, bias, trust — all addressed
- [ ] **Risk:** Guardrails layered (server → prompt → output filter → monitoring)
- [ ] **Risk:** Monitoring dashboards and structured logging in place

The Agent Builder's Playbook · Why → What → How → Considerations · Built with the Time-Off Agent